GAME DESIGN DOCUMENT

Sky Shooter: Save the City

Revised Game Design & Systems Sample

Prepared by: Nagesh Ranbhise
Role: Game Designer • Unity Developer • 2D/3D Digital Artist
Purpose: Portfolio sample demonstrating practical game design, systems thinking, combat design, progression planning and implementation-oriented documentation.

Revision note: This version removes the unrelated Souls-like examples, ground-boss mechanics, Resolve meter, Skeleton XP examples and Time-Rewind ability from the previous sample. It focuses on the aerial combat, city-defense and enemy-battlefield identity of Sky Shooter: Save the City.

1. GAME OVERVIEW

Genre: 2D mobile arcade shooter using 3D assets
Platform: Android
Engine: Unity

High Concept

The player pilots a combat aircraft through increasingly dangerous battle environments, engages enemy forces, avoids incoming threats, uses primary weapons and lock-on missiles, and progresses through themed levels while defending the city and surviving enemy battlefields.

Core Player Loop

Enter battlefield → control aircraft → locate and engage enemy forces → evade enemy attacks → use weapons and missile lock-on → destroy threats → complete the level objective → unlock/progress to the next level or theme → replay to improve performance.

Design Pillars

- Readable aerial combat: enemy attacks, projectiles, targeting and impacts should be understandable during fast movement.
- Responsive mobile control: aircraft movement and targeting must feel predictable and controllable with a mobile joystick.
- Combat decision-making: positioning, target priority and missile use should matter, rather than relying only on raw damage.
- Escalating battlefield pressure: later encounters should increase challenge through enemy behavior, formations, attack pressure and level progression.
- Clear progression: players should understand which levels/themes are available and what they need to do to advance.

2. PLAYER AIRCRAFT & CONTROLS

Aircraft Movement

The player controls a combat aircraft using joystick-based movement. The movement system is designed for a 2D gameplay presentation while using a 3D aircraft asset. Movement, rotation/tilt response, playable-area boundaries and screen positioning are gameplay parameters that can be tuned during balancing.

Control Goals

- Immediate response to joystick input.
- Predictable aircraft movement during combat.
- Prevent accidental movement outside the playable battlefield.
- Maintain enough maneuverability to evade projectiles and enemy attacks.
- Keep targeting and movement understandable on a mobile screen.

Player State & Damage

The aircraft can receive combat damage through enemy interactions. Damage feedback should communicate clearly when the player is hit, allowing the player to understand why health/survivability has changed and respond through movement or target prioritization.

3. WEAPONS & TARGETING

Primary Weapon

The primary weapon provides reliable repeated damage against enemy targets. Fire rate, projectile speed, damage, hit response and related values are balancing variables. The system should reward positioning and target selection rather than making weapon power the only determinant of success.

Lock-On Missile System

The lock-on missile system is a deliberate targeting mechanic that gives the player a stronger attack option after acquiring a target. The documented implementation values are: **lock duration = 0.5 seconds; maximum lock distance = 20 units; lock angle = 35 degrees.** These values should remain exposed as balancing variables rather than permanent assumptions.

Targeting Flow

Detect eligible enemy → evaluate distance/angle → begin lock → maintain lock for required duration → confirm target → launch missile → resolve hit/damage → provide impact feedback.

Combat Feedback

- Lock confirmation should be visually distinct.
- Missile launch should be immediately readable.
- Enemy impact should communicate hit and damage.
- Enemy destruction should provide stronger feedback than a normal hit.
- Player damage should have clear visual/audio/UI feedback.

4. ENEMY BATTLEFIELD DESIGN

The enemy battlefield is the central pressure system of Sky Shooter. Enemy encounters should create different tactical problems for the player while remaining readable within a fast aerial combat environment.

Enemy Roles

Enemy role	Gameplay purpose	Player response
Basic Shooter	Creates direct ranged pressure.	Move, evade and prioritize the threat.
Bomber / Area Threat	Creates dangerous zones or burst pressure.	Maintain position awareness and destroy quickly.
Formation Enemy	Controls battlefield space through group movement.	Read formation, reposition and select targets.
Elite Enemy	Creates a higher-priority combat threat.	Use target priority, primary weapon and missile system.

Enemy Encounter Principles

- Use enemy placement and movement to create pressure from different directions.
- Avoid visual clutter that makes projectiles or targets impossible to read.
- Introduce stronger enemy pressure progressively rather than presenting maximum difficulty immediately.
- Use target priority to encourage decisions during combat.
- Enemy attacks should provide enough visual information for a skilled player to react.

Boss / Major Enemy Direction

For Sky Shooter, major encounters should remain consistent with the aerial battlefield theme. Boss or major-enemy attacks should therefore be based on aircraft movement, missile/projectile pressure, formations, attack runs, area bombardment or other aerial threats—not ground-character attacks such as punches, grabs or ground AoE smashes.

5. SAVE THE CITY — MISSION & LEVEL STRUCTURE

The title and fantasy of the game should be reflected directly in the player's objectives. The player is not simply fighting enemies in an abstract arena; the combat is framed as a defense mission against enemy forces.

Mission Experience

- Enter the current battlefield/theme.
- Engage enemy forces and survive incoming attacks.
- Prioritize dangerous targets and use available weapons.
- Complete the level's combat/defense objective.
- Receive completion/progression feedback.
- Unlock or progress toward the next level/theme.

Themed Progression

The project uses multiple themed progression categories, including City, Battle and Sea, with a TrainingSession used for player onboarding/tutorial-style interaction. The progression system tracks theme-specific level unlock information.

Progression Data

- CityUnlockedLevel
- BattleUnlockedLevel
- SeaUnlockedLevel

- Handedness / player control preference
- LevelProgressTracker_MultiTheme

6. TRAINING & PLAYER ONBOARDING

The TrainingSession provides a controlled environment for teaching the player the basic interaction model before or alongside more demanding combat. The purpose is to reduce cognitive load when the player first encounters the full battlefield.

Training Priorities

- Aircraft movement using the mobile joystick.
- Understanding the relationship between movement and aiming.
- Basic attack interaction.
- Understanding enemy targeting and threat recognition.
- Introducing the missile lock-on concept.
- Providing clear feedback for successful actions.

Training should teach the minimum required actions first and allow the player to understand the combat language before encountering higher battlefield pressure.

7. UI, MOBILE PRESENTATION & PLAYER FEEDBACK

Mobile UI Principles

- Controls must remain accessible without covering important combat information.
- Critical buttons should have clear visual hierarchy.
- The player should distinguish gameplay controls from informational UI.
- Safe-area handling should prevent important controls or information from being obscured on different devices.
- Feedback for lock-on, missile launch, damage and enemy defeat should be readable during action.

Input System

The project uses Unity's input-system workflow for player interaction. Mobile joystick control is central to aircraft movement, while gameplay/UI interaction should be designed so important actions remain accessible without conflicting input behavior.

8. DIFFICULTY & BALANCING APPROACH

Balancing should focus on the actual aerial combat loop rather than importing unrelated RPG or Souls-like progression systems.

Primary balancing variables

- Player movement speed and maneuverability.
- Enemy movement and attack frequency.
- Enemy spawn timing and density.
- Enemy health and damage.
- Projectile speed and visibility.
- Primary weapon fire rate and damage.
- Missile lock duration, distance and angle.
- Level length and encounter pacing.
- Player survivability and recovery opportunities.

Balancing goal

The player should feel increasingly capable as they learn movement, targeting and enemy patterns, while later battlefields continue to create meaningful pressure. Difficulty should come from readable threats and tactical decisions—not unfair or visually ambiguous attacks.

9. DESIGNER → PROGRAMMER HANDOFF

A mechanic is implementation-ready when its behavior can be communicated without subjective instructions such as “make it cool.” Each system should define:

- Player flow: inputs, states, transitions and expected outcomes.
- Numbers: duration, cooldown, range, damage, speed, limits and thresholds.
- Visual/audio tells: what the player sees or hears and when.
- Edge cases: unusual interactions that need explicit handling.
- Acceptance criteria: objective conditions that define completion.
- Balancing variables: values exposed for iteration.

Example — Missile Lock

Input/target detection → check target eligibility → maintain lock for 0.5 s → require target within 20-unit distance and 35-degree lock angle → confirm lock → allow missile launch → resolve target hit/damage → play feedback. Values remain tunable for balancing.

10. IMPLEMENTATION & PROJECT OWNERSHIP

Sky Shooter: Save the City is an independently developed Unity mobile project. The development work spans game concept, gameplay systems, 2D/3D content, Unity scene setup, C# scripting, UI, testing, optimization, Android build/deployment and publishing.

Selected implemented/production areas

- Unity gameplay development and C# scripting.
- 3D aircraft asset used within the 2D gameplay presentation.
- Player movement and mobile joystick controls.
- Enemy spawning and combat behavior.
- Primary shooting and missile systems.
- Lock-on targeting and aim/target feedback.
- Player/enemy damage interactions.
- Multi-theme level progression.
- TrainingSession onboarding environment.
- Mobile UI and safe-area handling.
- Android build, testing and Google Play deployment.
- AdMob monetization integration.

11. DESIGN REVIEW & FUTURE ITERATION

This document describes the current design direction and implementation-oriented structure. Future iteration should be based on observed player behavior, testing results and the actual feature set of each released level.

- Review whether enemy encounters create clear but escalating pressure.
- Measure whether missile lock feels useful without eliminating positioning.
- Check whether the player understands why damage is received.
- Review level pacing and enemy density between themes.
- Validate that progression communicates what has been unlocked and what comes next.
- Use playtesting to identify confusing controls, difficulty spikes and underused systems.

12. DESIGN SUMMARY

Sky Shooter: Save the City is designed around an accessible but increasingly demanding aerial combat loop. The player's core skills are movement, threat recognition, target prioritization, weapon use and battlefield positioning. The game's city-defense identity is reinforced through enemy battlefields and themed progression.

The design documentation connects player experience with implementation detail: measurable combat parameters, readable targeting feedback, enemy roles, progression structure, mobile controls, balancing variables and programmer handoff criteria. This makes the GDD useful both as a game-design sample and as documentation for a real Unity project.

Nagesh Ranbhise
Game Designer • Unity Developer • 2D/3D Digital Artist